ASSIGNMENT 1 – CO1

ARTIFICIAL INTELLIGENCE – CSA17

TASK 1 - Source code (BFS)

```
from collections import deque
import time
import matplotlib.pyplot as plt
def create_graph(n):
    graph = [[] for _ in range(n)]

    for i in range(n):
        for j in (2*i+1, 2*i+2):
            if j < n:
                graph[i].append(j)
                graph[j].append(i)
    return graph
def bfs(graph, start, goal):
    queue = deque([start])
    visited = {start}
    parent = {}
    expanded = 0
    while queue:
        node = queue.popleft()
        expanded += 1

        if node == goal:
            break
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                parent[neighbour] = node
                queue.append(neighbour)
    if goal not in visited:
        return [], expanded
    path = [goal]
    while path[-1] != start:
        path.append(parent[path[-1]])

    return path[::-1], expanded
sizes = [1000, 10000, 100000]
times = []
expansions = []
print("BREADTH FIRST SEARCH")
print("--------------------")
for n in sizes:
    graph = create_graph(n)
    start_time = time.perf_counter()
    path, expanded = bfs(graph, 0, n-1)
    elapsed = (time.perf_counter()-start_time)*1000
    times.append(elapsed)
    expansions.append(expanded)
```

```
    print("\nStates:", n)
    print("Start: Pharmacy (0)")
    print("Goal: Ward", n-1)
    print("Path:", path)
    print("Expanded Nodes:", expanded)
    print("Execution Time:", round(elapsed, 3), "ms")

plt.plot(sizes, times, marker='o')
plt.xlabel("Number of States")
plt.ylabel("Execution Time (ms)")
plt.title("BFS Performance Analysis")
plt.grid()
plt.savefig("BFS_Graph.png")
plt.show()

print("\nBFS execution completed.")
```

Actual output

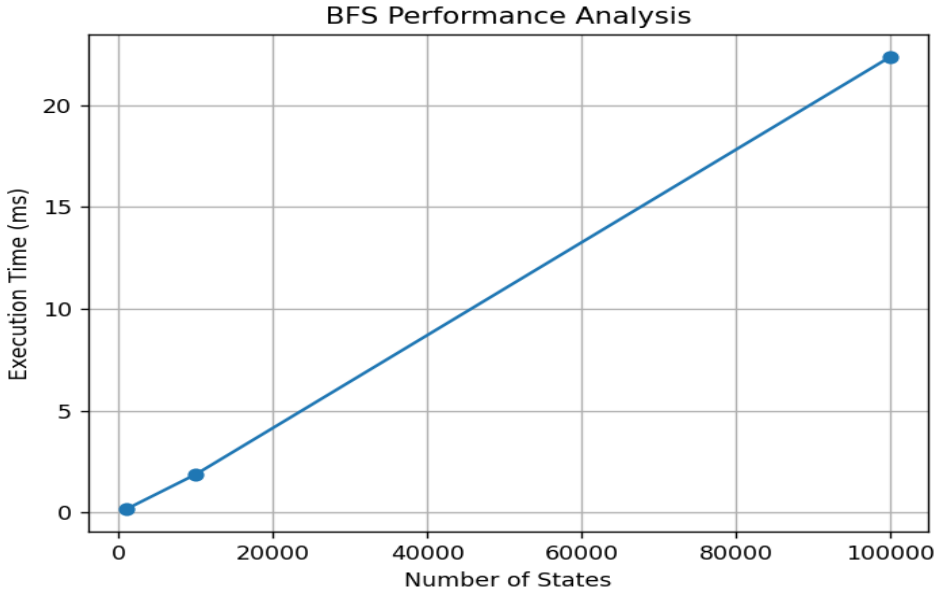
```
BREADTH FIRST SEARCH
--------------------

States: 1000
Start: Pharmacy (0)
Goal: Ward 999
Path: [0, 2, 6, 14, 30, 61, 124, 249, 499, 999]
Expanded Nodes: 1000
Execution Time: 0.204 ms

States: 10000
Start: Pharmacy (0)
Goal: Ward 9999
Path: [0, 1, 3, 8, 18, 38, 77, 155, 311, 624, 1249, 2499, 4999, 9999]
Expanded Nodes: 10000
Execution Time: 2.015 ms

States: 100000
Start: Pharmacy (0)
Goal: Ward 99999
Path: [0, 2, 5, 11, 23, 47, 96, 194, 389, 780, 1561, 3124, 6249, 12499, 24999,
9999, 99999]
Expanded Nodes: 100000
Execution Time: 22.324 ms
```

Graph



Source code (DFS)

```
import time
import matplotlib.pyplot as plt
def create_graph(n):
    graph = [[] for _ in range(n)]
    for i in range(n):
        for j in (2*i+1, 2*i+2):
            if j < n:
                graph[i].append(j)
                graph[j].append(i)
    return graph
def dfs(graph, start, goal):
    visited = set()
    parent = {}
    expanded = 0
    def search(node):
        nonlocal expanded
        visited.add(node)
        expanded += 1
        if node == goal:
            return True
        for neighbour in graph[node]:
            if neighbour not in visited:
                parent[neighbour] = node
                if search(neighbour):
                    return True
        return False
    if not search(start):
        return [], expanded
    path = [goal]
    while path[-1] != start:
        path.append(parent[path[-1]])
    return path[::-1], expanded
sizes = [1000, 10000, 100000]
times = []
expansions = []
print("DEPTH FIRST SEARCH")
print("------------------")
for n in sizes:
    graph = create_graph(n)
    start_time = time.perf_counter()
    path, expanded = dfs(graph, 0, n-1)
    elapsed = (time.perf_counter()-start_time)*1000
    times.append(elapsed)
    expansions.append(expanded)
    print("\nStates:", n)
    print("Start: Pharmacy (0)")
    print("Goal: Ward", n-1)
```

```
    print("Path:", path)
    print("Expanded Nodes:", expanded)
    print("Execution Time:", round(elapsed, 3), "ms")
plt.plot(sizes, times, marker='o')
plt.xlabel("Number of States")
plt.ylabel("Execution Time (ms)")
plt.title("DFS Performance Analysis")
plt.grid()
plt.savefig("DFS_Graph.png")
plt.show()
print("\nDFS execution completed.")
```

Actual output

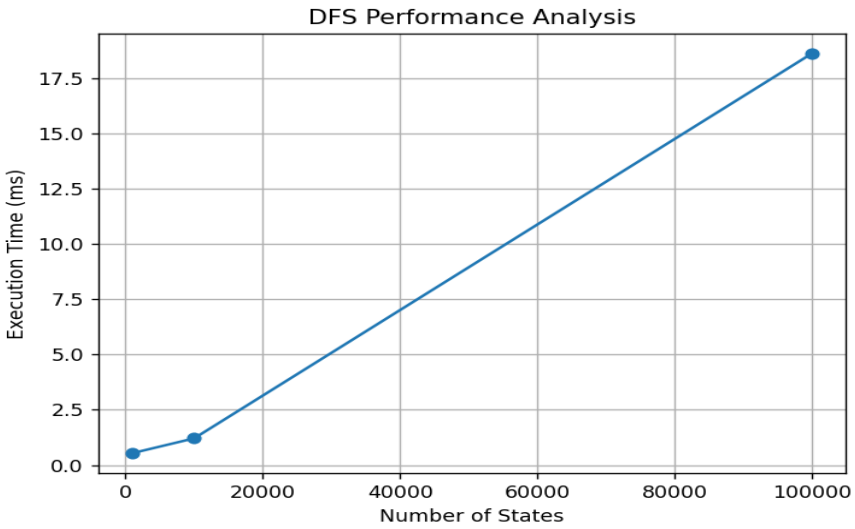
```
DEPTH FIRST SEARCH
------------------

States: 1000
Start: Pharmacy (0)
Goal: Ward 999
Path: [0, 2, 6, 14, 30, 61, 124, 249, 499, 999]
Expanded Nodes: 981
Execution Time: 0.54 ms

States: 10000
Start: Pharmacy (0)
Goal: Ward 9999
Path: [0, 1, 3, 8, 18, 38, 77, 155, 311, 624, 1249, 2499, 4999, 9999]
Expanded Nodes: 3626
Execution Time: 1.21 ms

States: 100000
Start: Pharmacy (0)
Goal: Ward 99999
Path: [0, 2, 5, 11, 23, 47, 96, 194, 389, 780, 1561, 3124, 6249, 12499, 24999,
9999, 99999]
Expanded Nodes: 68940
Execution Time: 18.597 ms
```

Graph



Source Code (UCS)

```
import heapq
import time
import matplotlib.pyplot as plt
def create_graph(n):
    graph = [[] for _ in range(n)]
    for i in range(n):
        for j in (2*i+1, 2*i+2):
            if j < n:
                cost = 1 + (j % 7)
                graph[i].append((j, cost))
                graph[j].append((i, cost))
    graph[0].append((n-1, n*10))
    graph[n-1].append((0, n*10))
    return graph
def ucs(graph, start, goal):
    pq = [(0, start)]
    best = {start: 0}
    parent = {}
    expanded = 0
    while pq:
        cost, node = heapq.heappop(pq)
        if cost != best[node]:
            continue
        expanded += 1
        if node == goal:
            break
        for neighbour, edge_cost in graph[node]:
            new_cost = cost + edge_cost
            if new_cost < best.get(neighbour, float('inf')):
                best[neighbour] = new_cost
                parent[neighbour] = node
                heapq.heappush(pq, (new_cost, neighbour))
    if goal not in best:
        return [], expanded, 0
    path = [goal]
    while path[-1] != start:
        path.append(parent[path[-1]])
    return path[::-1], expanded, best[goal]
sizes = [1000, 10000, 100000]
times = []
expansions = []
print("UNIFORM COST SEARCH")
print("-------------------")
for n in sizes:
    graph = create_graph(n)
    start_time = time.perf_counter()
    path, expanded, cost = ucs(graph, 0, n-1)
```

```
    elapsed = (time.perf_counter()-start_time)*1000
    times.append(elapsed)
    expansions.append(expanded)
    print("\nStates:", n)
    print("Start: Pharmacy (0)")
    print("Goal: Ward", n-1)
    print("Path:", path)
    print("Total Travel Cost:", cost)
    print("Expanded Nodes:", expanded)
    print("Execution Time:", round(elapsed, 3), "ms")
plt.plot(sizes, times, marker='o')
plt.xlabel("Number of States")
plt.ylabel("Execution Time (ms)")
plt.title("UCS Performance Analysis")
plt.grid()
plt.savefig("UCS_Graph.png")
plt.show()
print("\nUCS execution completed.")
```

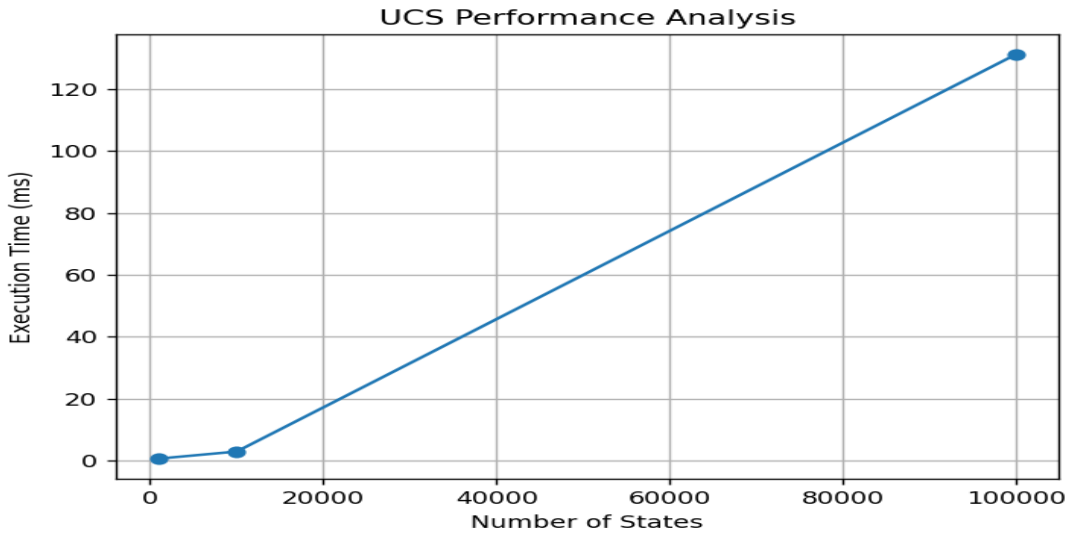
Actual output

```
UNIFORM COST SEARCH
-------------------

States: 1000
Start: Pharmacy (0)
Goal: Ward 999
Path: [0, 2, 6, 14, 30, 61, 124, 249, 499, 999]
Total Travel Cost: 40
Expanded Nodes: 868
Execution Time: 0.678 ms

States: 10000
Start: Pharmacy (0)
Goal: Ward 9999
Path: [0, 1, 3, 8, 18, 38, 77, 155, 311, 624, 1249, 2499, 4999, 9999]
Total Travel Cost: 37
Expanded Nodes: 2138
Execution Time: 2.959 ms

States: 100000
Start: Pharmacy (0)
Goal: Ward 99999
Path: [0, 2, 5, 11, 23, 47, 96, 194, 389, 780, 1561, 3124, 6249, 12499, 24999, 4
9999, 99999]
Total Travel Cost: 73
Expanded Nodes: 95243
Execution Time: 131.194 ms
```



Task 2 – Identify the Basic Operation

Source code

```
from collections import deque
hospital = {
    "Pharmacy": ["Junction A"],
    "Junction A": ["Pharmacy", "Junction B", "Junction C"],
    "Junction B": ["Junction A", "Ward 1"],
    "Junction C": ["Junction A", "Lift"],
    "Lift": ["Junction C", "Ward 2"],
    "Ward 1": ["Junction B"],
    "Ward 2": ["Lift"]
}
def bfs(start, goal):
    queue = deque([[start]])
    visited = set()

    while queue:
        path = queue.popleft()
        location = path[-1]

        if location == goal:
            return path

        if location not in visited:
            visited.add(location)

            for next_location in hospital[location]:
                if next_location not in visited:
                    queue.append(path + [next_location])

    return None

start = "Pharmacy"
goal = "Ward 2"

route = bfs(start, goal)

print("Starting Location:", start)
print("Destination:", goal)
print("Robot Route:", " -> ".join(route))
print("Total Moves:", len(route) - 1)
```

Output

```
-
Starting Location: Pharmacy
Destination: Ward 2
Robot Route: Pharmacy -> Junction A -> Junction C -> Lift -> Ward 2
Total Moves: 4
```

Task 8 – Implement and Test the Algorithms

Implementation

```
import heapq
import time
from collections import deque
def generate_graph(n):
    graph = [[] for _ in range(n)]
    for i in range(n):
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n:
            graph[i].append((left, 1))
            graph[left].append((i, 1))
        if right < n:
            graph[i].append((right, 2))
            graph[right].append((i, 2))
    return graph
def bfs(graph, start, goal):
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if node == goal:
            return path
        for neighbour, cost in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append((neighbour, path + [neighbour]))
    return None
def dfs(graph, node, goal, visited=None, path=None):
    if visited is None:
        visited = set()
        path = []
    visited.add(node)
    path.append(node)
    if node == goal:
        return path.copy()
    for neighbour, cost in graph[node]:
        if neighbour not in visited:
            result = dfs(
                graph, neighbour, goal, visited, path
            )
            if result is not None:
                return result
    path.pop()
    return None
def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    best cost = {start: 0}
```

```
        if node == goal:
            return path, cost

        if cost > best_cost[node]:
            continue

        for neighbour, weight in graph[node]:
            new_cost = cost + weight

            if new_cost < best_cost.get(neighbour, float("inf")):
                best_cost[neighbour] = new_cost
                heapq.heappush(
                    pq,
                    (new_cost, neighbour, path + [neighbour])
                )

    return None, float("inf")
sizes = [1000, 10000, 100000, 1000000]

for n in sizes:
    print("\nHospital Graph Size:", n)
    graph = generate_graph(n)
    start = 0
    goal = n - 1
    print("Start:", start)
    print("Goal:", goal)
    t1 = time.perf_counter()
    path_bfs = bfs(graph, start, goal)
    t2 = time.perf_counter()

    print("BFS Path Found:", path_bfs is not None)
    print("BFS Time:", round(t2 - t1, 6), "seconds")

    t1 = time.perf_counter()
    path_dfs = dfs(graph, start, goal)
    t2 = time.perf_counter()

    print("DFS Path Found:", path_dfs is not None)
    print("DFS Time:", round(t2 - t1, 6), "seconds")

    t1 = time.perf_counter()
    path_ucs, cost_ucs = ucs(graph, start, goal)
    t2 = time.perf_counter()

    print("UCS Path Found:", path_ucs is not None)
    print("UCS Path Cost:", cost_ucs)
    print("UCS Time:", round(t2 - t1, 6), "seconds")
```

Output

```
Hospital Graph Size: 1000
Start: 0
Goal: 999
BFS Path Found: True
BFS Time: 0.000764 seconds
DFS Path Found: True
DFS Time: 0.000228 seconds
UCS Path Found: True
UCS Path Cost: 14
UCS Time: 0.001273 seconds

Hospital Graph Size: 10000
Start: 0
Goal: 9999
BFS Path Found: True
BFS Time: 0.004569 seconds
DFS Path Found: True
DFS Time: 0.000794 seconds
UCS Path Found: True
UCS Path Cost: 17
UCS Time: 0.009564 seconds

Hospital Graph Size: 100000
Start: 0
Goal: 99999
BFS Path Found: True
BFS Time: 0.047051 seconds
DFS Path Found: True
DFS Time: 0.015765 seconds
UCS Path Found: True
UCS Path Cost: 21
UCS Time: 0.067958 seconds

Hospital Graph Size: 1000000
Start: 0
Goal: 999999
BFS Path Found: True
BFS Time: 0.957571 seconds
DFS Path Found: True
```

Task 9 – Record Experimental Performance

Source code

```
import math
import time
import heapq
import csv
import matplotlib.pyplot as plt
SIZES = [1000, 10000, 100000, 1000000]
SEED = 42
RUNS = 5
def neighbours(node, n):
    rows = math.ceil(math.sqrt(n))
    cols = math.ceil(n / rows)
    r, c = divmod(node, cols)
    result = []
    positions = [
        (r - 1, c),
        (r, c + 1),
        (r + 1, c),
        (r, c - 1)
    ]
    for nr, nc in positions:
        nxt = nr * cols + nc
        if 0 <= nr < rows and 0 <= nc < cols and nxt < n:
            result.append(nxt)
    return result
def edge_cost(a, b):
    x = min(a, b)
    y = max(a, b)
    value = (x * 73856093 + y * 19349663 + SEED) % 1000003
    return 1 + value % 9
def get_path(parent, start, goal):
    if goal not in parent:
        return None
    path = []
    node = goal
    while node != start:
        path.append(node)
        node = parent[node]
    path.append(start)
    path.reverse()
    return path
def bfs(n, start, goal):
    from collections import deque
    queue = deque([start])
    parent = {start: None}
    generated = 1
    expanded = 0
    max_frontier = 1
```

```
    while stack:
        node = stack.pop()
        expanded += 1
        if node == goal:
            break
        for nxt in reversed(neighbours(node, n)):
            if nxt not in parent:
                parent[nxt] = node
                stack.append(nxt)
                generated += 1
        max_frontier = max(max_frontier, len(stack))
    path = get_path(parent, start, goal)
    return path, generated, expanded, max_frontier
def ucs(n, start, goal):
    counter = 0
    pq = [(0, counter, start)]

    best_cost = {start: 0}
    parent = {start: None}

    generated = 1
    expanded = 0
    max_frontier = 1
    while pq:
        cost, _, node = heapq.heappop(pq)
        if cost != best_cost[node]:
            continue
        expanded += 1
        if node == goal:
            break
        for nxt in neighbours(node, n):
            new_cost = cost + edge_cost(node, nxt)

            if new_cost < best_cost.get(nxt, float("inf")):
                best_cost[nxt] = new_cost
                parent[nxt] = node
                counter += 1
                heapq.heappush(pq, (new_cost, counter, nxt))
                generated += 1
        max_frontier = max(max_frontier, len(pq))
    path = get_path(parent, start, goal)

    return path, generated, expanded, max_frontier
def calculate_cost(path):
    if path is None:
        return None
```

```
    return total
results = []

algorithms = {
    "BFS": bfs,
    "DFS": dfs,
    "UCS": ucs
}
for n in SIZES:
    start = 0
    goal = n - 1
    print("\nHospital states:", n)
    for name, algorithm in algorithms.items():
        times = []
        last_result = None

        for run in range(RUNS):
            t1 = time.perf_counter()
            path, generated, expanded, frontier = algorithm(
                n, start, goal
            )
            t2 = time.perf_counter()
            times.append(t2 - t1)
            last_result = (path, generated, expanded, frontier)
        path, generated, expanded, frontier = last_result
        depth = len(path) - 1 if path else None
        cost = calculate_cost(path)
        if name == "UCS":
            optimal = True
        else:
            _, _, _, _ = ucs(n, start, goal)
            optimal = None
        results.append({
            "States": n,
            "Algorithm": name,
            "Generated": generated,
            "Expanded": expanded,
            "Max_Frontier": frontier,
            "Depth": depth,
            "Path_Cost": cost,
            "Average_Time": sum(times) / RUNS,
            "Optimal_Cost": optimal
        })
        print(
            name,
            "| Generated:", generated,
            "| Expanded:", expanded,
```

```
            "| Expanded:", expanded,
            "| Max Frontier:", frontier,
            "| Depth:", depth,
            "| Cost:", cost,
            "| Average Time:", round(sum(times) / RUNS, 6)
        )
for n in SIZES:
    ucs_cost = next(
        row["Path_Cost"] for row in results
        if row["States"] == n and row["Algorithm"] == "UCS"
    )

    for row in results:
        if row["States"] == n:
            row["Optimal_Cost"] = (
                row["Path_Cost"] == ucs_cost
            )
with open("task9_results.csv", "w", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=results[0].keys())
    writer.writeheader()
    writer.writerows(results)
metrics = [
    ("Generated", "Nodes Generated"),
    ("Expanded", "Nodes Expanded"),
    ("Max_Frontier", "Maximum Frontier Size"),
    ("Average_Time", "Average Execution Time")
]
for column, label in metrics:
    plt.figure()
    for name in algorithms:
        x = [r["States"] for r in results if r["Algorithm"] == name]
        y = [r[column] for r in results if r["Algorithm"] == name]

        plt.plot(x, y, marker="o", label=name)
    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Number of Hospital States")
    plt.ylabel(label)
    plt.title(label + " Comparison")
    plt.legend()
    plt.grid(True)
    plt.savefig("task9_" + column + ".png")
    plt.show()
print("\nResults saved in task9_results.csv")
print("Performance graphs saved as PNG files.")
print("Random seed:", SEED)
print("Number of runs:", RUNS)
```

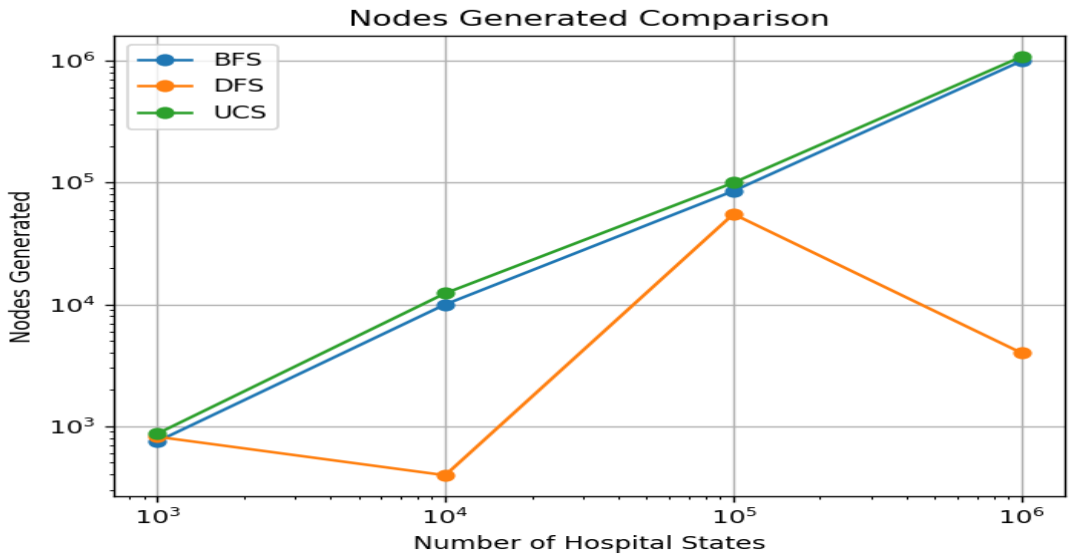
Actual output

```
Hospital states: 1000
BFS | Generated: 747 | Expanded: 724 | Max Frontier: 32 | Depth: 38 | Cost: 197
| Average Time: 0.000888
DFS | Generated: 819 | Expanded: 423 | Max Frontier: 397 | Depth: 422 | Cost: 21
07 | Average Time: 0.000605
UCS | Generated: 866 | Expanded: 685 | Max Frontier: 62 | Depth: 42 | Cost: 121
| Average Time: 0.001815

Hospital states: 10000
BFS | Generated: 10000 | Expanded: 10000 | Max Frontier: 100 | Depth: 198 | Cost
: 1017 | Average Time: 0.010559
DFS | Generated: 395 | Expanded: 199 | Max Frontier: 197 | Depth: 198 | Cost: 10
17 | Average Time: 0.000264
UCS | Generated: 12344 | Expanded: 9999 | Max Frontier: 234 | Depth: 198 | Cost:
 488 | Average Time: 0.037349

Hospital states: 100000
BFS | Generated: 85465 | Expanded: 85294 | Max Frontier: 316 | Depth: 459 | Cost
: 2292 | Average Time: 0.103409
DFS | Generated: 55269 | Expanded: 27722 | Max Frontier: 27548 | Depth: 27721 |
Cost: 138690 | Average Time: 0.043743
UCS | Generated: 99889 | Expanded: 81496 | Max Frontier: 525 | Depth: 521 | Cost
: 1528 | Average Time: 0.300414

Hospital states: 1000000
BFS | Generated: 1000000 | Expanded: 1000000 | Max Frontier: 1000 | Depth: 1998
| Cost: 9984 | Average Time: 1.360735
DFS | Generated: 3995 | Expanded: 1999 | Max Frontier: 1997 | Depth: 1998 | Cost
: 9984 | Average Time: 0.002757
UCS | Generated: 1084230 | Expanded: 1000000 | Max Frontier: 1561 | Depth: 1998
| Cost: 7128 | Average Time: 3.830418
```



Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

Source code

```
possible_k = [2, 4, 8]

print("Belief-State Experiment")
print("-----------------------")

for k in possible_k:

    # Without local sensor
    without_sensor = k

    # With perfect local sensor
    with_sensor = 1

    # Number of possible subsets
    possible_beliefs = 2 ** k

    print("\nPossible locations:", k)

    print("Without Sensor:")
    print("Belief states:", without_sensor)

    print("With Local Sensor:")
    print("Belief states:", with_sensor)

    print("Possible subsets:", possible_beliefs)

    print("Plan length without sensor:", k)
    print("Plan length with sensor:", 1)
```

Actual Output

```
Belief-State Experiment
-----------------------

Possible locations: 2
Without Sensor:
Belief states: 2
With Local Sensor:
Belief states: 1
Possible subsets: 4
Plan length without sensor: 2
Plan length with sensor: 1

Possible locations: 4
Without Sensor:
Belief states: 4
With Local Sensor:
Belief states: 1
Possible subsets: 16
Plan length without sensor: 4
Plan length with sensor: 1

Possible locations: 8
Without Sensor:
Belief states: 8
With Local Sensor:
Belief states: 1
Possible subsets: 256
Plan length without sensor: 8
Plan length with sensor: 1
```